PIP-Link

高性能远程桌面控制系统 - 客户端

License MIT | python 3.8+ | platform Windows | Linux

基于*UDP*的低延迟视频流传输 + *100Hz*高频控制指令发送

特性 • 快速开始 •系统架构 • 配置说明 • 开发文档

📖 项目简介

PIP-Link 是一个专为远程桌面控制设计的客户端应用程序，与基于ROS2的服务端配合使用。系统采用**TCP**握手 **+ UDP**视频流 **+ UDP**控制指令的混合架构，实现了：

- 🎥 低延迟视频传输: UDP分包传输，支持动态调整JPEG质量和分辨率
- 🎮 高频控制发送: 100Hz控制指令发送频率，支持鼠标(速度控制+5键+滚轮) + 键盘(71键同时检测)
- 🖥️ 自定义**UI**框架: 基于OpenCV + Pygame实现的完整UI系统
- ⚙️ 实时参数调整: 支持视频质量、图像调整(曝光/对比度/伽马)、鼠标灵敏度等参数的动态修改
- 📊 传输质量监控: 实时显示帧率、延迟、丢包率、带宽等指标

✨ 特性

核心功能

功能模块	说明
视频流接收	UDP协议接收JPEG压缩视频帧，支持自动重组和丢包处理
控制指令发送	100Hz频率发送鼠标速度、按键状态，支持F5快捷键切换Ready/Not Ready模式

功能模块	说明
参数同步	实时接收服务端流参数和客户端列表
图像调整	客户端可请求调整服务端的曝光度(0.1-3.0)、对比度(0.1-3.0)、伽马值(0.1-3.0)
配置持久化	JSON格式保存用户设置(连接信息、视频质量、控制参数等)

输入设备支持

鼠标控制

- 速度控制: 基于相对位移计算速度，支持灵敏度调节(0.1-5.0)
- 按键支持: 左键、右键、中键、Mouse4(侧键)、Mouse5(侧键)
- 滚轮支持: 向上滚动、向下滚动
- 速度限幅: 可配置最大/最小速度阈值

键盘控制

- 多键检测: 使用pynput实现，支持最多71个按键同时按下
- 按键映射: 涵盖功能键(F1-F12)、数字键、字母键、符号键、修饰键等
- 状态编码: 10字节bitmap编码，高效传输

UI功能

- 选项卡式调试面板: 7个功能选项卡(连接/流/客户端/统计/显示/图像/控制)
- 窗口模式: 支持窗口模式和无边框全屏模式切换
- 分辨率切换: 预设8种常用分辨率(4:3、16:9、16:10)
- 状态指示器: 左下角动态显示Ready/Not Ready状态，带呼吸灯效果
- 实时统计: 显示帧率、延迟、丢包率、带宽等传输指标

🚀 快速开始

环境要求

- 操作系统：Windows 10+ / Linux (Ubuntu 20.04+)
- **Python**版本：3.8 或更高
- 依赖库：
    - pygame >= 2.5.0
    - opencv-python >= 4.8.0
    - numpy >= 1.24.0
    - pynput >= 1.7.6

安装步骤

1. 克隆仓库

```
git clone https://github.com/yourusername/pip-link.git
cd pip-link
```

1. 安装依赖

```
pip install -r requirements.txt
```

1. 运行客户端

```
python main.py
```

首次使用

1. 启动服务端：确保ROS2服务端已启动并监听TCP端口(默认8888)
2. 配置连接：在客户端Connection选项卡输入服务器IP和端口
3. 建立连接：点击Connect按钮
4. 激活控制：按F5键切换到Ready状态，光标将隐藏并锁定

快捷键

按键	功能
`ESC`	切换调试面板显示/隐藏
`F5`	切换Ready/Not Ready控制状态

🏗️ 系统架构

网络通信协议

```
客户端                                    服务端
  |                                         |
  |---------- TCP握手(端口8888) -------->|  (连接建立)
  |<--------- ACK + ClientID -----------|
  |                                         |
  |<========= UDP视频流(端口8889) ========|  (持续接收)
  |                                         |
  |<========= UDP参数包(端口8890) ========|  (1Hz广播)
  |                                         |
  |========> UDP控制包(端口8891) ========>|  (100Hz发送)
```

端口分配

端口	协议	用途	频率
8888	TCP	握手和命令	连接时
8889	UDP	视频流(服务端→客户端)	30-60 FPS
8890	UDP	参数包(服务端→客户端)	1 Hz
8891	UDP	控制包(客户端→服务端)	100 Hz

> 注意：端口号基于*TCP*基准端口偏移计算，可通过服务端配置修改

数据包格式

控制包 (42字节)

```
包头(20字节):
  - MAGIC(4): 'CTRL'
  - SEQ(4): 序列号
  - TIMESTAMP(8): 时间戳
  - STATE(1): 0=Not Ready, 1=Ready
  - RESERVED(3): 保留字段

数据(22字节,仅Ready状态):
  - MOUSE_VX(4): X轴速度(float)
  - MOUSE_VY(4): Y轴速度(float)
  - MOUSE_BUTTONS(2): 按键状态bitmap
  - KEYBOARD_STATE(10): 键盘状态bitmap
  - RESERVED(2): 对齐
```

视频包 (动态大小)

```
包头(8字节):
  - FRAME_ID(4): 帧ID
  - PACKET_IDX(2): 包序号
  - TOTAL_PACKETS(2): 总包数

数据部分:
  - JPEG_DATA: JPEG压缩数据片段(最大65000字节)
```

项目结构

```
pip-link/
├── core/
|   ├── app.py              # 应用主控制器
|   ├── config.py           # 全局配置
|   └── state.py            # 应用状态管理
├── network/
|   ├── tcp_conn.py         # TCP连接管理
|   ├── udp_conn.py         # UDP视频流接收
```

```
|   ├── params_receiver.py  # 参数包接收
|   ├── control_sender.py   # 控制指令发送
|   ├── control_packet.py   # 控制包编解码
|   ├── keyboard_encoder.py # 键盘状态编码
|   └── manager.py          # 网络管理器
├── ui/
|   ├── manager.py          # UI管理器
|   ├── components/         # UI组件(按钮/标签/输入框等)
|   └── tabs/               # 选项卡实现
├── utils/
|   ├── events.py           # 事件总线
|   └── config_manager.py   # 配置管理器
├── config.json             # 用户配置文件
├── main.py                 # 程序入口
└── README.md
```

⚙️ 配置说明

config.json

```json
{
  "connection": {
    "server_ip": "192.168.1.106",
    "server_port": "8888"
  },
  "stream": {
    "jpeg_quality": 80,      // JPEG质量(1-100)
    "frame_scale": 1.0       // 缩放比例(0.1-1.0)
  },
  "image": {
    "exposure": 1.0,         // 曝光度(0.1-3.0)
    "contrast": 1.0,         // 对比度(0.1-3.0)
    "gamma": 2.0             // 伽马值(0.1-3.0)
  },
  "control": {
    "mouse_sensitivity": 1.0 // 鼠标灵敏度(0.1-5.0)
  },
  "display": {
    "window_mode": "windowed",   // "windowed" | "fullscreen"
```

```
    "resolution_index": 6        // 分辨率索引(0-7)
  }
}
```

Config.py 关键参数

```
# 窗口
DEFAULT_WIDTH = 1024
DEFAULT_HEIGHT = 768

# 网络端口偏移
UDP_PORT_OFFSET = 1       # 视频流
PARAMS_PORT_OFFSET = 2    # 参数
CONTROL_PORT_OFFSET = 3   # 控制

# 控制发送频率
CONTROL_SEND_RATE = 100   # Hz

# 鼠标控制
MOUSE_SCALE_FACTOR = 0.01        # 速度缩放因子(固定)
MAX_MOUSE_VELOCITY = 720.0       # 速度上限(px/s)
MIN_MOUSE_VELOCITY = -720.0      # 速度下限(px/s)
```

🔧 开发文档

核心类说明

ApplicationController (core/app.py)

应用主控制器，负责整体流程协调

主要方法:

- `run()`: 主循环
- `_handle_key()`: 键盘事件处理
- `_handle_mouse_button()`: 鼠标按键处理

- `on_mouse_move()`: 鼠标移动处理

NetworkManager (network/manager.py)

网络通信管理器

主要方法:

- `connect()`: 建立连接
- `disconnect()`: 断开连接
- `send_quality_settings()`: 发送质量设置
- `send_image_adjustment()`: 发送图像调整参数

ControlSender (network/control_sender.py)

控制指令发送器，100Hz发送频率

主要方法:

- `update_mouse_position()`: 更新鼠标速度
- `update_mouse_buttons()`: 更新鼠标按键状态
- `set_sensitivity()`: 设置灵敏度
- `toggle_state()`: 切换Ready状态

KeyboardEncoder (network/keyboard_encoder.py)

键盘状态编码器，使用pynput监听

编码格式: 10字节bitmap，每个bit对应一个按键

自定义UI框架

基于OpenCV实现的轻量级UI框架

核心组件:

- `Object`: 基类，实现鼠标事件、焦点管理
- `Button`: 按钮，支持悬停/按下状态
- `Label`: 文本标签，支持多行文本和对齐
- `TextBox`: 文本输入框，支持光标移动、长按重复
- `Panel`: 面板容器
- `TabbedPanel`: 选项卡面板

事件系统:

```
# 订阅事件
event_bus.subscribe(Events.CONNECTED, callback)

# 发布事件
event_bus.publish(Events.CONNECTED)
```

添加新的控制功能

1. 修改控制包格式 (network/control_packet.py)
2. 更新编码器 (network/control_sender.py)
3. 服务端同步修改 (remote_link/control_packet.py)

添加新的UI选项卡

1. 在 `ui/tabs/` 创建新选项卡类
2. 在 `UIManager` 注册选项卡
3. 实现 `update()` 和 `get_components()` 方法

📊 性能指标

典型性能

指标	数值
视频延迟	15-30ms
控制延迟	<10ms
视频帧率	30-60 FPS
控制频率	100 Hz
丢包率	<1% (局域网)
带宽占用	5-15 Mbps (取决于质量设置)

优化建议

- 降低延迟：降低JPEG质量(50-70)、减小分辨率(frame_scale=0.8)
- 提高质量：JPEG质量80+、frame_scale=1.0
- 节省带宽：降低帧率(30 FPS)、压缩质量(50-60)

🤝 配套服务端

本客户端需要配合ROS2服务端使用。

服务端功能：

- 接收视频流并通过UDP分包发送
- 接收控制指令并发布到ROS2 topic
- 管理多客户端连接
- 提供参数广播服务

ROS2包：`remote_link`

服务端主要包含以下模块：

- `tcp_handshake_manager.py`：TCP握手管理
- `stream_manager.py`：视频流管理

- `udp_video_stream.py`: UDP视频发送
- `control_receiver.py`: 控制指令接收
- `params_sender.py`: 参数广播

📝 更新日志

v1.0.0 (2025-11-07)

- ✨ 初始版本发布
- 🎥 UDP视频流接收
- 🎮 100Hz控制指令发送
- 🖥️ 完整UI系统
- ⚙️ 配置持久化
- 📊 传输质量监控

🐛 已知问题

- 在某些Linux发行版上，pynput可能需要root权限
- 全屏模式切换时可能出现短暂黑屏
- 高丢包率环境下视频可能出现花屏

📄 许可证

本项目采用 MIT License

🙏 致谢

- Pygame - 窗口和事件系统
- OpenCV - 图像处理和UI绘制
- pynput - 键盘监听
- ROS2 - 机器人操作系统

📮 联系方式

- **Issues**: GitHub Issues
- **Email**: mabolong2006@outlook.com

⭐ 如果这个项目对你有帮助，请给个**Star!** ⭐

Made with ❤️ for Remote Desktop Control